

Anil Seth



Using Python to Manage MP3 Tags

The author loves to play around and experiment with software and share the results of his work with readers. He has wide and varied tastes and MP3 music is one of them.

It is nice to be able to play the songs you like more often.

Amarok was the first player I came across that allowed the creation of dynamic playlists using a rating tag. For example, you could have a playlist with half the songs with a rating of 4 or more and the remaining could be any choice from your library. This was many years ago, when it was Amarok version 3. The rating was kept in the database of the music player and I put in the effort to rate at least the songs I loved to hear often.

Somewhere down the line, the database was lost. I can't remember whether it happened when upgrading the Amarok player to version 4 or when I switched the desktop. Anyway, I never got around to setting the ratings again.

There is now an ID3 tag which stores the rating. However, it is far too tedious to change MP3 tags using a music player, especially if the music library is a mess, with inconsistent file and directory names.

Can you use the ID3 tags to make the library better organised?

You may decide to name each music file using the artist and the song's title. However, it is possible that not all songs in your library have the proper tags, especially if you have digitised them yourself from ancient CDs.

So, you could decide on the following strategy:

1. Create a CSV file with the file name and the selected ID3 tags as columns.
2. Use a spreadsheet to modify the columns.
3. Update the ID3 tags using the modified CSV file.
4. Rename all the MP3 files using the ID3 tags.

There are a number of Python modules to modify the tags in MP3 files. Two common packages are Mutagen and eyed3. Mutagen has the advantage that it works with multiple file types, including MP3, OGG and FLAC, while eyed3 is meant for MP3 files only.

Getting started

On Fedora, install the following packages:

```
$sudo dnf install python2-mutagen, python-eyed3
```

The following is the minimum code for running any of the three options. You pass the name of the option (*get_tags*, *update_tags* or *rename*) and the starting directory.

```
#!/usr/bin/python
import sys,os
from os.path import join
if __name__ == '__main__':
    try:
        option = eval(sys.argv[1])
        if len(sys.argv) > 2:
            path = sys.argv[2]
        else:
            path = '.'
        option(path)
    except:
        print("Parameters: one of [get_tags,update_
tags,rename] path(default .)")
```

Creating the CSV file

The basic code for *get_tags* is as follows. You iterate over the files and select each MP3 file. You will need to call the function *get_file_tags* to get the tags you need. This function is the one that will depend on the Python module you use. Two other versions are given later.

```
def get_mp3_file(path):
    """ Walk through all files and select mp3 files.
        Common for all options"""
    for root,dirnames,filenames in os.walk(path):
        for f in filenames:
            if len(f) > 4 and f[-4:].lower() == '.mp3':
                yield join(root,f)

def get_tags(path):
    """ Create a dictionary with file name as the key
        and values are a tuple of the desired tag values
        Save the dictionary as csv file suitable for a
        spreadsheet"""
    result = {}
    for fn in get_mp3_file(path):
        result[fn]=get_file_tags(fn)
    out=open('mp3.csv','w')
    for f in result:
        out.write("%s;%s;%s;%s\n"%((f,) + result[f]))
```

Shown below is the code for *get_file_tags* if you're using the Mutagen module: